

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

Jackfruit Phase-1

Software Requirements Specification
for Online Bookstore – Team 15

Item	Student Entry
Project Title	Online Bookstore – Web-based Book Ordering & Catalog Management System
Project ID	BOOKSTORE-2026-T15
Group Number	Team 15
Team Members	Anushka P – PES1202401127 / B S Keertana – PES1202400463 / Bhagath Naveen Mullakara – PES1202403272 / Bhuvan Raj H – PES1202502988
Course	UE24CS341A – Software Engineering
Technology Stack	MERN – MongoDB, Express.js, React, Node.js
Development Methodology	Agile
Version	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for Online Bookstore, a web-based application that lets a bookstore list its catalog online and lets customers browse, search, and purchase books. This SRS covers the full system: the customer-facing storefront and the Administrator / Staff dashboard.

1.2 Intended Audience and Reading Suggestions

This document is intended for the development team (all four members), the course instructor/evaluator, and anyone reviewing the project's requirements baseline. Section 2 gives context and scope; Section 3 covers interfaces; Section 4 covers use cases and diagrams; Section 5 lists detailed functional requirements; Sections 6–7 and the appendices cover non-functional, security, and supporting reference material.

1.3 Product Scope

Online Bookstore digitizes catalog browsing and order management for a small independent or campus bookstore. It replaces in-person, walk-in ordering with a self-service storefront for customers and a lightweight management dashboard for Administrator / Staff, without requiring a costly, over-engineered e-commerce platform.

1.4 References

- Team 15 Synopsis / Project Proposal (Online Bookstore), v1.0
- PES University – Jackfruit Software Engineering Project Guidelines, UE24CS341A
- PES University – Software Engineering Level 3 Student Submission Guide & Exemplar

2. Overall Description

2.1 Product Perspective

Online Bookstore is a new, self-contained web application; it is not a follow-on to an existing system. It consists of a React single-page frontend, a Node.js/Express REST API backend, and a MongoDB database, communicating over HTTPS.

2.2 Product Functions

- Customer account registration and login
- Catalog browsing, search, and book detail viewing
- Cart management, checkout, and order placement
- Order status tracking and order history
- Administrator / Staff catalog management (add/edit/disable books, update stock)
- Administrator / Staff order management (view orders, update fulfillment status)

2.3 User Classes and Characteristics

User Class	Privileges	Characteristics
Customer	Browse, search, order, complete mock payment, and track own orders	Non-technical, occasional use
Administrator / Staff	Manage catalog and orders	Higher privilege; store staff

2.4 Operating Environment

The application runs as a responsive web app accessible from modern desktop and mobile browsers (Chrome, Firefox, Edge). The backend runs on Node.js and connects to a MongoDB instance (local or MongoDB Atlas).

2.5 Design and Implementation Constraints

- Technology stack is fixed to MERN (MongoDB, Express.js, React, Node.js) — the team's chosen stack over the alternative Django option
- Agile methodology is mandatory for this course; work is organized into sprints with a visible backlog
- Git/GitHub must be used with real, incremental commit history from every member
- CI/CD pipeline, code reviews, and security validation are mandatory course deliverables
- Authentication must use JWT-based sessions

2.6 Assumptions and Dependencies

- A real payment gateway is out of scope; a mock/simulated payment service is assumed acceptable
- The catalog will be seeded with sample book data for development and testing
- A stable internet connection and a MongoDB instance (local or Atlas) are available to all team members during development

3. External Interface Requirements

3.1 User Interfaces

The customer storefront includes: home/catalog page, search results, book detail page, cart, checkout, and order history. The Administrator / Staff dashboard includes: catalog management and order management views. Both are responsive and built with React.

3.2 Software Interfaces

- Frontend (React) communicates with the backend via a REST API (JSON over HTTPS)
- Backend (Node.js/Express) communicates with MongoDB via the Mongoose ODM
- Backend communicates with a mock/simulated Payment Service for order payment authorization

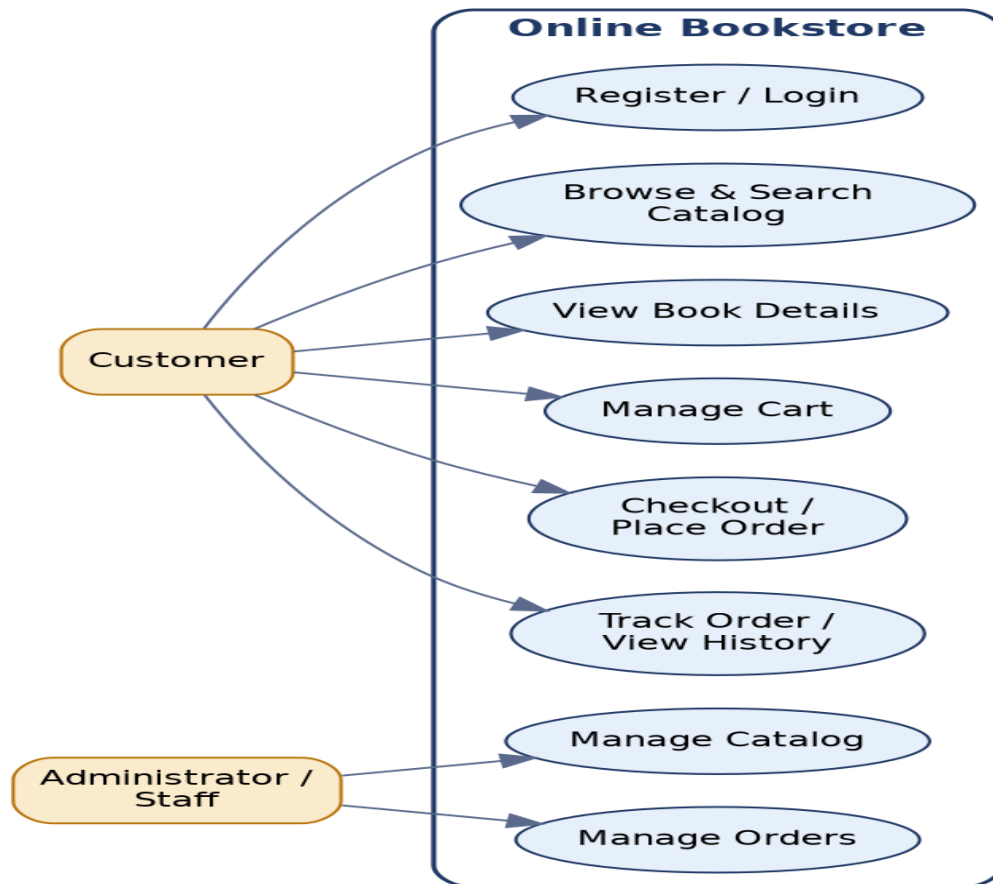
3.3 Communications Interfaces

All client-server communication uses HTTPS. Authenticated requests carry a JWT bearer token in the Authorization header. Data is exchanged as JSON.

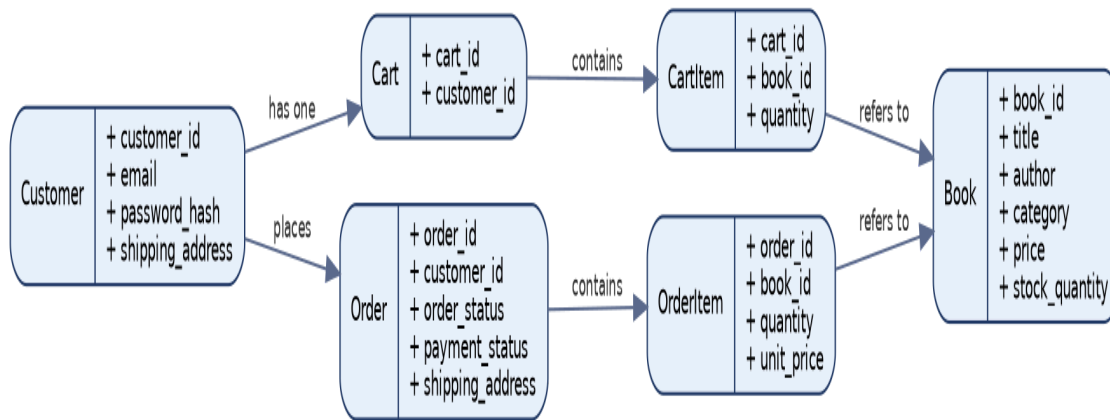
4. Analysis Models — Use Cases and Diagrams

Use Case ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Register / Login	Customer	Application available	Authenticated session
UC-02	Browse & Search Catalog	Customer	None	Matching books displayed
UC-03	View Book Details	Customer	Book exists	Book detail page shown
UC-04	Manage Cart	Customer	Logged in	Cart updated
UC-05	Checkout / Place Order	Customer	Logged in; valid cart	Order created
UC-06	Track Order / View History	Customer	Order exists	Current status displayed
UC-07	Manage Catalog	Administrator / Staff	Administrator/Staff authenticated	Catalog updated
UC-08	Manage Orders	Administrator / Staff	Administrator/Staff authenticated	Order status updated

4.1 Use Case Diagram



4.2 Domain / Data Model Diagram



The domain model above covers the Customer, Cart, CartItem, Book, Order, and OrderItem entities and their relationships, matching the field layout in Appendix B.

5. System Features — Functional Requirements

ID	Requirement	Priority	Verification
FR-001	The system shall allow a customer to register an account using a unique email address and a password.	High	Functional test
FR-002	The system shall allow a registered customer to log in using their email and password.	High	Functional test
FR-003	The system shall allow a customer to browse the book catalog without requiring login.	High	Functional test
FR-004	The system shall allow a customer to search the catalog by title, author, or category.	High	Functional test
FR-005	The system shall display a book detail page showing title, author, category, price, stock status, description, and cover image.	High	Functional test
FR-006	The system shall allow a logged-in customer to add an in-stock book to their cart.	High	Functional test
FR-007	The system shall allow a customer to update the quantity of, or remove, an item in their cart.	High	Functional test
FR-008	The system shall calculate the cart total from item price and quantity for all items in the cart.	High	Unit + system test
FR-009	The system shall allow a customer to place an order only after all mandatory checkout information (shipping address, cart contents) is valid.	High	Negative + positive test
FR-010	The system shall assign a unique order number to every successfully created order.	High	Integration test
FR-011	The system shall display order status as PLACED, PROCESSING, SHIPPED, DELIVERED, or CANCELLED.	High	System test
FR-012	The system shall allow a customer to view their own order history and current order status.	High	Functional test
FR-013	The system shall allow Administrator / Staff to add, edit, or disable book listings in the catalog.	Medium	Functional test
FR-014	The system shall allow Administrator / Staff to update stock quantity for a book.	Medium	Functional test
FR-015	The system shall allow Administrator / Staff to view incoming orders and update their fulfillment status.	High	Functional test
FR-016	The system shall record payment success or failure against the corresponding order using a	High	Integration test

	mock/simulated payment service and shall not confirm an order when payment fails.		
--	---	--	--

6. Other Nonfunctional Requirements

6.1–6.2 Performance, Safety & Availability

Safety: Online Bookstore is a standard e-commerce web application with no physical, medical, or life-safety implications; no safety-critical requirements apply beyond the standard security requirements in Section 6.3.

ID	Category	Measurable Requirement	Verification
NFR-001	Performance	95% of catalog search requests shall receive a response within 1.5 seconds under a load of 50 concurrent users.	Performance test
NFR-002	Availability	The application shall be available at least 99% during the defined operating window, excluding planned maintenance.	Operational review
NFR-003	Usability	A new customer shall be able to place a basic order in no more than 5 primary screens.	Usability walkthrough
NFR-004	Reliability	An order shall not be created twice when a customer retries checkout after a payment timeout.	Negative/integration test
NFR-005	Maintainability	Business logic shall be separated into identifiable service modules with automated unit tests.	Code review
NFR-006	Testability	Core services shall expose testable interfaces without requiring the production payment service.	Unit/integration test

6.3 Security Requirements

ID	Security Requirement	Validation Approach
SEC-001	Passwords shall not be stored in plaintext; they shall be hashed before storage.	Code/configuration review
SEC-002	Role-based access shall prevent customers from accessing Administrator/Staff-only endpoints.	Authorization tests
SEC-003	The system shall validate and sanitize all user-supplied fields before processing.	Negative/security tests
SEC-004	Authenticated APIs shall reject expired or invalid JWT tokens.	Security test
SEC-005	Sensitive payment information shall not be stored unnecessarily by the application.	Design review
SEC-006	Security-relevant events such as failed login and unauthorized access attempts shall be logged.	System test

6.4 Software Quality Attributes

- Maintainability is prioritized over raw performance for this course project, given the short timeline
- Usability: checkout must be completable without prior training or instructions

6.5 Business Rules / Domain Rules

- Only in-stock books can be added to a cart or order
- A customer may cancel or modify an order only while its status is PLACED
- Only Administrator / Staff can add, edit, or disable catalog items
- Only Administrator / Staff can update an order's fulfillment status
- An order must contain at least one valid item

- A mock payment failure shall not result in a confirmed order

7. Other Requirements

No additional database, legal, or localization requirements beyond those already listed are identified at this stage.

Appendix A: Glossary

- Cart — a customer's temporary, unordered selection of books before checkout
- Catalog — the full set of books available for browsing and purchase
- Checkout — the process of converting a cart into a confirmed order
- RBAC — Role-Based Access Control, restricting actions by user role (Customer vs. Administrator / Staff)
- JWT — JSON Web Token, used to authenticate API requests
- Stock / Inventory — the quantity of a given book currently available for sale
- Category — a book classification used for browsing and filtering the catalog (e.g., Fiction, Non-fiction, Academic)
- Mock/Simulated Payment — a stand-in payment step that returns a success or failure result without contacting a real payment gateway

Appendix B: Field Layouts

Field	Length	Data Type	Description	Mandatory
customer_id	36	ObjectId/String	Unique customer identifier	Y
shipping_address	300	String	Customer delivery address	Y
email	100	String	Unique login email	Y
password_hash	60	String	Hashed customer password	Y
book_id	36	ObjectId/String	Unique book identifier	Y
title	150	String	Book title	Y
author	100	String	Book author	Y
category	50	String	Book category/genre (e.g., Fiction, Academic)	Y
price	10,2	Decimal	Current selling price	Y
stock_quantity	6	Integer	Units currently in stock	Y
order_id	36	ObjectId/String	Unique order identifier	Y
order_status	20	Enum/String	PLACED/PROCESSING/SHIPPED /DELIVERED/CANCELLED	Y
payment_status	20	Enum/String	Mock payment result	Y

Appendix C: Requirement Traceability Matrix (Initial)

Architecture, design, and code references will be completed once Deliverable 3 (Design) is finalized; this initial matrix records requirement IDs against planned test coverage.

Requirement ID	Brief Description	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
FR-001	Customer registration	TBD	TBD	TBD	TBD	TBD
FR-006	Add to cart	TBD	TBD	TBD	TBD	TBD
FR-009	Place order	TBD	TBD	TBD	TBD	TBD
SEC-002	Role-based access	TBD	TBD	TBD	TBD	TBD
NFR-001	Search performance	TBD	TBD	TBD	TBD	TBD

Part B: Initial Validation Specification

At this requirements stage, validation planning is focused on how each requirement category will eventually be validated; this is not the final test report.

Requirement Category	Planned Validation	Example
Functional	Black-box functional tests	Valid/invalid checkout, cart edits, search filters
Security	Authentication/authorization + security testing	Customer attempts an Administrator/Staff-only endpoint
Performance	Load/response-time test	Concurrent catalog search and checkout requests
Usability	Task-based walkthrough	New customer places their first order
Reliability	Failure/retry scenarios	Payment timeout and retry does not duplicate order
Unit	Automated unit tests + coverage	Cart total calculation, stock validation

Part C: Project Plan

Work Breakdown Structure

Work Package	Tasks	Owner	Effort (person-days)	Output
WP1	Requirements + use cases	All	4	SRS
WP2	Architecture + UML	All (led by Keertana & Bhuvan)	4	Architecture/Design
WP3	Catalog & search module	Anushka P	6	Working feature
WP4	Cart & checkout module	B S Keertana	6	Working feature
WP5	Auth module (registration/login)	Bhagath Naveen Mullakara	5	Working feature
WP6	Admin catalog & order management module	Bhuvan Raj H	7	Working feature
WP7	Testing + CI/CD + security	All	6	Test/validation evidence
WP8	Final validation + report	All	4	Final report

Tool Stack

Activity	Tool	Evidence to Submit
Version control	Git + GitHub	Repository URL, commit history
Frontend	React (Create React App / Vite)	Component source, build output
Backend	Node.js + Express	API source, route documentation
Database	MongoDB (via Mongoose ODM)	Schema definitions, seed data
Agile planning	GitHub Projects / Jira	Backlog, user stories, sprint board
CI/CD	GitHub Actions	Pipeline run logs / screenshots
Containerization	Docker	Dockerfile + build/run evidence
Testing	Jest + Supertest (backend), React Testing Library (frontend)	Test reports
Static analysis	SonarQube or similar / ESLint	Analysis report
API testing	Postman	Collection/results
Design	Draw.io / PlantUML	UML diagrams

Agile lifecycle justification: the team will work in six sprints (see the synopsis's short-term plan), each producing a demonstrable increment, with a GitHub Projects/Jira backlog of user stories mapped to the functional requirements above.

SRS Addendum: Section 6.3 Security Objectives

This addendum completes Section 6.3 (Security Requirements) of the Online Bookstore SRS v1.0. The security requirements SEC-001 to SEC-006 are listed in Section 6.3 of the SRS; the security objectives they serve are stated below. Together they form the complete security section (objectives and requirements).

6.3.1 Security Objectives

ID	Security Objective	Related requirements
SO-1	Confidentiality of credentials: user passwords can never be recovered from the database or logs.	SEC-001, SEC-005
SO-2	Authenticity of requests: only holders of a valid, unexpired token can use protected APIs.	SEC-004
SO-3	Authorisation (least privilege): customers can reach only their own cart and orders; catalog and fulfilment functions are limited to Administrator / Staff.	SEC-002
SO-4	Integrity of data: untrusted input cannot alter queries, scripts or stored data; orders and payment results cannot be duplicated or forged.	SEC-003, NFR-004
SO-5	Accountability and detection: security-relevant events are recorded so misuse can be detected and investigated.	SEC-006
SO-6	Data minimisation: the application never receives or stores real payment card data.	SEC-005

6.3.2 Security Requirements

Security requirements SEC-001 to SEC-006 are as specified in Section 6.3 of the SRS. Their validation approaches are planned in the Software Test Plan, Section 5.1 (Security Validation), and their realisation is described in the Software Architecture and Design Specification, Section 3.9 (Security Architecture).

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Test Plan (STP)**Online Bookstore – Team 15**

Project	Online Bookstore – Web-based Book Ordering & Catalog Management System
Project ID	BOOKSTORE-2026-T15
Version	1.0
Authors	Team 15: Anushka P, B S Keertana, Bhagath Naveen Mullakara, Bhuvan Raj H
Date	01-10-2026
Status	Final

1. Introduction

Purpose: This document defines the test plan for the Online Bookstore v1.0. It states the test objectives, scope, strategy, environment, schedule, responsibilities and the test cases that validate the Software Requirements Specification (SRS v1.0) and the Software Architecture and Design Specification (SAD v1.0).

Scope: Testing covers customer registration/login, catalog browsing and search, book details, cart management, checkout with the mock payment, order tracking and history, Administrator / Staff catalog and order management, and the non-functional and security requirements. Real payment gateways, shipping carriers and MongoDB/Node.js internals are excluded.

References: Online Bookstore SRS v1.0; Online Bookstore SAD v1.0; PES University SE Level 3 Submission Guide; OWASP Top 10 and ASVS.

Definitions: SRS (Software Requirements Specification), SAD (Software Architecture and Design), RTM (Requirements Traceability Matrix), UC (use case), FR / NFR / SEC (functional, non-functional, security requirement), JWT, RBAC, TC (test case).

2. Test Items

- Authentication module (C-04) and security middleware (C-03)
- Catalog and search module (C-05)
- Cart module (C-06)
- Order / checkout module (C-07) and Payment Adapter with mock payment service (C-08, C-12)
- Admin catalog and order management module (C-09)
- React customer storefront (C-01) and Administrator / Staff dashboard (C-02)
- Data access layer and MongoDB schemas (C-10, C-11); logging (C-13)

3. Features to be Tested

Features are mapped to SRS requirement IDs (full mapping in Section 13):

Requirement	Feature	Use case
FR-001, FR-002	Registration and login	UC-01

Requirement	Feature	Use case
FR-003, FR-004	Browse and search catalog	UC-02
FR-005	Book detail page	UC-03
FR-006, FR-007, FR-008	Cart add / update / remove and total	UC-04
FR-009, FR-010, FR-016	Checkout, unique order number, mock payment result	UC-05
FR-011, FR-012	Order status and order history	UC-06
FR-013, FR-014	Admin catalog add / edit / disable and stock update	UC-07
FR-015	Admin order list and fulfilment status update	UC-08
NFR-001	Search: 95% of requests within 1.5 s at 50 concurrent users	UC-02
NFR-002	Availability of at least 99% in the operating window	All
NFR-003	New customer can order in no more than 5 primary screens	UC-05
NFR-004	No duplicate order when checkout is retried after a payment timeout	UC-05
NFR-005, NFR-006	Maintainability and testability (service modules, stubbable payment)	All
SEC-001 to SEC-006	Password hashing, RBAC, input validation, JWT checks, payment data minimisation, security logging	All

4. Features Not to be Tested

- Real payment gateway behaviour (only the mock/simulated payment service is in scope)
- MongoDB, Node.js, Express and React framework internals (assumed tested by their maintainers)
- Hosting provider infrastructure and network reliability beyond the defined test environment
- Shipping carrier integration, e-mail/SMS notifications (not part of v1.0)
- Browsers other than current Chrome, Firefox and Edge

5. Test Approach / Strategy

Levels:

- **Unit tests** (Jest) for service-level logic: cart total, validation, order state machine, payment adapter mapping, idempotency. Repositories and the payment adapter are mocked.
- **Integration tests** (Jest + Supertest against an in-memory or test MongoDB): API ↔ service ↔ database flows, RBAC and error codes, payment stub outcomes.
- **System tests** (Cypress UI automation plus Postman collections): end-to-end customer and admin journeys through the UI and API.
- **Acceptance tests (UAT)**: walkthrough of the use cases with the instructor/evaluator using seeded data.

Types: functional (black-box), regression (full automated suite on every pull request in GitHub Actions), performance (JMeter), usability (task-based walkthrough), reliability (timeout/retry/failure scenarios), white-box review and coverage (Jest coverage report, code review).

Test design techniques: equivalence partitioning and boundary value analysis (quantities, address length, password length, page limit), state-transition testing (order status), negative testing (invalid input, missing auth), and decision-table style checks for checkout rules.

Entry criteria: stable build on the main branch with CI green; seed data and test accounts available; test environment ready; relevant SRS/SAD baselined.

Exit criteria: 100% of planned test cases executed; no open critical or high defects; all high-priority requirements covered by at least one passed test; unit coverage report produced; all acceptance criteria satisfied.

5.1 Security Validation

Security validation is planned against SEC-001 to SEC-006 and the threats in SAD Section 3.9 (STRIDE). Executed evidence is stored in the security-validation folder of the repository.

Security area	Validation activity	Requirement	Test cases
Password storage	Inspect DB and logs; confirm bcrypt hashes, no plaintext	SEC-001	TC-AUTH-01, TC-AUTH-08
Authentication / session	Expired, tampered and missing JWT tests; login with wrong password	SEC-004	TC-AUTH-05, TC-AUTH-06, TC-SEC-04, TC-CART-03
Authorization (RBAC)	Customer token against every /admin endpoint; cross-customer order access	SEC-002	TC-SEC-01, TC-SEC-02, TC-TRK-03
Input validation	NoSQL operator injection, script injection, over-length and malformed fields (fuzzing of text inputs with OWASP ZAP and Postman)	SEC-003	TC-CAT-06, TC-SEC-03, TC-ORD-03, TC-ADM-02
Payment data	Schema and log review for card data	SEC-005	TC-SEC-05
Logging and abuse control	Failed login and unauthorised access events logged; login rate limit	SEC-006	TC-SEC-06, TC-SEC-07
Transport and headers	HTTPS-only access, TLS 1.2+, helmet security headers checked with ZAP baseline scan	SAD 3.9	ZAP baseline report
Dependencies and code	npm audit, ESLint security rules and SonarQube (or similar) scan in CI	SAD 3.9	CI evidence
Penetration testing	Manual attempts on authentication, authorization and checkout flows (privilege escalation, token reuse, IDOR on order ids, price tampering)	SEC-002, SEC-004, FR-009	Pen-test checklist and findings log

6. Test Environment

Item	Details
Hardware	Developer laptops and one cloud/VM test instance; modern desktop and mobile browsers
Software	Online Bookstore v1.0 (React + Node.js/Express), MongoDB (test database, local or Atlas), mock payment service, Docker
Tools	Jest + Supertest, React Testing Library, Cypress (UI automation), Postman/Newman (API), Apache JMeter (performance), OWASP ZAP (security scan), GitHub Actions (CI), SonarQube or similar, GitHub Projects/Jira (defect tracking)
Test data	Seed script: 1,000 sample books across Fiction, Non-fiction and Academic; accounts for 3 customers and 1 Administrator / Staff; payment stub that can return SUCCESS, DECLINED or TIMEOUT on demand

7. Test Schedule

Planned milestones (dates to be aligned with the team's sprint calendar and the faculty deadlines):

Milestone	Planned date
Test plan and test case design	05-Oct-2026

Milestone	Planned date
Test environment and seed data ready	08-Oct-2026
Unit testing alongside development (each sprint)	From 12-Oct-2026
Integration and system test execution	26-Oct-2026 to 14-Nov-2026
Security validation and performance test	09-Nov-2026 to 14-Nov-2026
UAT and final regression	16-Nov-2026 to 18-Nov-2026
Test summary and validation report	20-Nov-2026

8. Test Deliverables

- Test Plan (this document) and test cases (Appendix A; also maintained as test-cases.xlsx)
- Automated test scripts (Jest, Supertest, Cypress), Postman collection, JMeter plan
- Test data and seed scripts
- Test execution logs with Actual Result and Pass/Fail filled in
- Unit test report and coverage report; CI pipeline evidence
- Security validation evidence (ZAP report, pen-test findings)
- Defect reports and Test Summary Report; updated RTM

9. Roles and Responsibilities

Role	Name	Responsibility
Test Engineer – Authentication and security	Bhagath Naveen Mullakara	Tests for C-03/C-04 (TC-AUTH, TC-SEC); runs security validation
Test Engineer – Catalog and search	Anushka P	Tests for C-05 (TC-CAT); performance test TC-PERF-01
Test Engineer – Cart and checkout	B S Keertana	Tests for C-06/C-07/C-08 (TC-CART, TC-ORD, TC-TRK, TC-TST)
Test Engineer – Admin	Bhuvan Raj H	Tests for C-09 (TC-ADM); CI test pipeline
All members	Team 15	Fix defects in own modules, review each other's tests, join UAT and usability sessions
Product Owner / sign-off	Course instructor / evaluator	Approve results and readiness

10. Risks and Mitigation

Risk	Mitigation
Delay in stable builds across four module owners	Merge to main only with passing CI; weekly integration build
Test environment or database downtime	Containerised environment that can be recreated in minutes; backup cloud VM
Payment timeout and concurrency scenarios hard to reproduce	Controllable payment stub (SUCCESS / DECLINED / TIMEOUT) and scripted concurrent requests
Performance test hardware differs from deployment	Report results against NFR-001 with the hardware stated; repeat on the deployed instance
Limited time for security testing	Automate ZAP baseline and npm audit in CI; reserve a dedicated security testing slot

11. Assumptions & Dependencies

- Mock payment service and stub are available before checkout testing begins.
- Seed data and test accounts are provided before execution.
- MongoDB (local or Atlas) and the CI runners are available to all members.
- The SRS and SAD are baselined; changes follow reviewed pull requests and update the RTM.

12. Suspension & Resumption Criteria

Suspend testing if:

- Environment or database unavailable for more than 4 hours
- Build is so unstable that more than 30% of planned test cases are blocked
- A critical defect in login or checkout blocks all dependent tests

Resume testing when: blocking defects are fixed and verified by a smoke test, and the environment is stable again.

13. Test Case Management & Traceability

Test cases are identified as TC-<AREA>-<nn> (AUTH, CAT, CART, ORD, TRK, ADM, SEC, PERF, AVAIL, USA, MNT, TST) and listed in Appendix A with level, requirement IDs, steps, test data and expected result. The RTM below links each SRS requirement to its architecture component and design element (SAD v1.0) and to its test cases, so that every requirement leads to a design element and a test.

13.1 Requirements Traceability Matrix

Req ID	Description	Component(s)	Design ref	Test cases
FR-001	Register account	C-04, C-10	API-01	TC-AUTH-01, 02, 03, 07
FR-002	Login	C-04, C-03	API-02, SD-3	TC-AUTH-04, 05, 07
FR-003	Browse without login	C-05	API-03	TC-CAT-01
FR-004	Search title/author/category	C-05, C-10	API-03	TC-CAT-02, 03, 04, 05, 06, 07
FR-005	Book detail page	C-05	API-04	TC-CAT-08 to 15
FR-006	Add in-stock book to cart	C-06, C-05	API-06	TC-CART-01, 02, 03, TC-CAT-10, 15
FR-007	Update/remove cart item	C-06	API-07, API-08	TC-CART-04, 05, 06
FR-008	Cart total	C-06	API-05	TC-CART-07, 08
FR-009	Valid checkout only	C-07, C-03	API-09, SD-1	TC-ORD-01, 02, 03, 04, 08, 09, 10
FR-010	Unique order number	C-07, C-10	SD-1 step 12	TC-ORD-04, 05, 09, 10
FR-011	Order status values	C-07, C-09	State machine (Fig 4.1), SD-2	TC-TRK-01, 04, TC-ADM-07, 08, 09, 14
FR-012	Order history/status	C-07	API-10, API-11	TC-TRK-01, 02, 03, 07, 08, TC-ADM-09
FR-013	Add/edit/disable books	C-09, C-05	API-13, API-14	TC-ADM-01, 02, 03, 04, 10, 11
FR-014	Update stock	C-09	API-15	TC-ADM-05
FR-015	View orders, update status	C-09, C-07	API-16, API-17, SD-2	TC-ADM-06, 07, 08, 09, 12, 13, 14
FR-016	Payment result recorded	C-07, C-08, C-12	SD-1	TC-ORD-04, 06, 07, 10
NFR-001	Search performance	C-05, C-10	Indexes, pagination	TC-PERF-01, TC-CAT-07
NFR-002	Availability	C-13	API-18	TC-AVAIL-01

Req ID	Description	Component(s)	Design ref	Test cases
NFR-003	<= 5 screens to order	C-01	SAD 4.5	TC-USA-01
NFR-004	No duplicate order on retry	C-07, C-08, C-10	ADR-03, SD-1	TC-ORD-07
NFR-005	Maintainability	All modules	SAD 3.5	TC-MNT-01
NFR-006	Testability	C-07, C-08	ADR-05	TC-TST-01
SEC-001	Password hashing	C-04	SAD 3.9	TC-AUTH-01, 08
SEC-002	RBAC	C-03, C-09	SD-2	TC-SEC-01, 02, TC-TRK-03
SEC-003	Validate and sanitise input	C-03	SAD 3.9	TC-AUTH-03, TC-CAT-06, TC-ORD-03, TC-ADM-02, TC-SEC-03
SEC-004	Reject invalid/expired JWT	C-03	SAD 3.9	TC-AUTH-06, TC-CART-03, TC-TRK-08, TC-SEC-04
SEC-005	No unnecessary payment data	C-07, C-08, C-12	SO-6	TC-SEC-05
SEC-006	Security event logging	C-13, C-03	SD-2, SD-3	TC-AUTH-05, TC-SEC-06, 07

13.2 Use-case coverage

Use case	Name	Requirements	Test cases	Count
UC-01	Register / Login	FR-001, FR-002, SEC-001, SEC-004	TC-AUTH-01 to 08	8
UC-02	Browse & Search Catalog	FR-003, FR-004, NFR-001	TC-CAT-01 to 07, TC-PERF-01	8
UC-03	View Book Details	FR-005	TC-CAT-08 to 15	8
UC-04	Manage Cart	FR-006, FR-007, FR-008	TC-CART-01 to 08	8
UC-05	Checkout / Place Order	FR-009, FR-010, FR-016, NFR-004	TC-ORD-01 to 10	10
UC-06	Track Order / View History	FR-011, FR-012	TC-TRK-01 to 08	8
UC-07	Manage Catalog	FR-013, FR-014, SEC-002	TC-ADM-01 to 05, 10, 11, TC-SEC-01	8
UC-08	Manage Orders	FR-015, FR-011, SEC-002	TC-ADM-06 to 09, 12 to 14, TC-SEC-02	8

Total planned test cases: 75 (unit, integration, system, security, performance, usability, review). Every use case has at least 8 test cases (8 to 10 per use case, as per the submission guide), in addition to the cross-cutting security and non-functional cases.

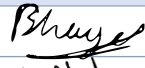
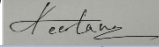
14. Test Metrics & Reporting

Metrics collected:

- % of test cases executed and % passed / failed
- Requirement coverage (requirements with at least one passed test)
- Unit test line and branch coverage for service modules
- Defect density, defect aging and severity distribution
- Performance results versus NFR-001; availability versus NFR-002

Reports: daily execution status during test cycles, CI results on every pull request, and the final Test Summary Report with the updated RTM.

15. Approvals

Role	Name	Signature / Date
Team Member (Authentication)	Bhagath Naveen Mullakara	 2-10-2026
Team Member (Catalog & Search)	Anushka P	 02-10-2026
Team Member (Cart & Checkout)	B S Keertana	 02-10-2026
Team Member (Admin)	Bhuvan Raj H	 02-10-2026
Product Owner / Instructor		

Appendix A: Test Cases

Levels: Unit, Integration, System, Security, Performance, Usability, Review (inspection). Actual Result and Result are filled in after manual or automated execution; A dash means not yet executed. Test data uses dummy accounts and seeded books only.

Test Case ID	Level	Requirement	Description	Precondition	Steps	Test Data	Expected Result	Actual	Result
TC-AUTH-01	Unit	FR-001, SEC-001	Register with valid, unique email	Email not in DB	Call AuthService.register()	name=Test User; email=reader1@example.com; password=Passw0rd!23	User saved; response 201; stored passwordHash is a 60-char bcrypt hash, not the plaintext	—	Pass / Fail
TC-AUTH-02	Unit	FR-001	Reject duplicate email	reader1@example.com already registered	Register again with same email	email=reader1@example.com	409 EMAIL_EXISTS; no second user record	—	Pass / Fail
TC-AUTH-03	Unit	FR-001, SEC-003	Reject invalid email format and short password (boundary)	None	Register with bad inputs; then with 7-char and 8-char passwords	email=reader.example.com; passwords Abcdef1 and Abcdefg1	422 VALIDATION_ERROR for bad email and 7 chars; 8 chars accepted	—	Pass / Fail
TC-AUTH-04	Integration	FR-002	Login with valid credentials	Registered user exists	POST /auth/login	reader1@example.com / Passw0rd!23	200; JWT returned with role=CUSTOMER and expiry of 1 hour	—	Pass / Fail
TC-AUTH-05	Integration	FR-002, SEC-006	Login with wrong password	Registered user exists	POST /auth/login with wrong password	reader1@example.com / Wrong#123	401 INVALID_CREDENTIALS (generic message); failed-login event written to security log without the password	—	Pass / Fail
TC-AUTH-06	Integration	SEC-004	Expired token rejected	Token with exp in the past	GET /cart with expired token	Expired JWT	401 TOKEN_EXPIRED; no cart data returned	—	Pass / Fail
TC-AUTH-07	System	FR-001, FR-002	Register then login end-to-end in the UI	App running	Open Register; submit; open Login; submit	New email + valid password	Customer lands on catalog, logged in; logout returns to anonymous state	—	Pass / Fail
TC-AUTH-08	Review	SEC-001	Passwords never stored or logged in plaintext	Several users registered	Inspect users collection, application logs and source	Test accounts	Only bcrypt hashes in DB; no password text in logs or code	—	Pass / Fail
TC-CAT-01	System	FR-003	Browse catalog without login	Seed data loaded; not logged in	Open home page	None	Active books listed with title, author, price; no login prompt	—	Pass / Fail
TC-CAT-02	Integration	FR-004	Search by title	Book 'Clean Code' exists	GET /books?q=clean code	q=clean code	200; results contain 'Clean Code'; unrelated titles absent	—	Pass / Fail

Test Case ID	Level	Requirement	Description	Precondition	Steps	Test Data	Expected Result	Actual	Result
TC-CAT-03	Integration	FR-004	Search by author	Books by one author exist	GET /books?q=<author name>	q=Robert Martin	200; only that author's books returned	—	Pass / Fail
TC-CAT-04	Integration	FR-004	Filter by category	Books in Fiction and Academic	GET /books?category=Academic	category=Academic	200; every item has category Academic	—	Pass / Fail
TC-CAT-05	Integration	FR-004	Search with no match	Seed data loaded	GET /books?q=zzzxqj	q=zzzxqj	200 with empty items; UI shows 'No books found'	—	Pass / Fail
TC-CAT-06	Unit	FR-004, SEC-003	Operator/regex injection in search is neutralised	Seed data loaded	GET /books with injection payloads	q[\$ne]=; q=.*(+	No server error; payload treated as plain text; no unintended match-all result	—	Pass / Fail
TC-CAT-07	Integration	FR-004	Pagination limit boundary	More than 60 books seeded	GET /books?limit=50 then limit=51	limit=50 / 51	50 items returned; limit=51 gives 422 (or is capped at 50)	—	Pass / Fail
TC-CAT-08	System	FR-005	Book detail page shows all fields	Book exists with cover image	Click a book card	Any seeded book	Title, author, category, price, stock status, description and cover image displayed	—	Pass / Fail
TC-CAT-09	Integration	FR-005	Unknown or disabled book returns 404	One disabled book exists	GET /books/{unknownId}; GET /books/{disabledId}	Random ObjectId; disabled book id	404 BOOK_NOT_FOUND for both (customer view)	—	Pass / Fail
TC-CAT-10	System	FR-005, FR-006	Out-of-stock book is clearly marked	Book with stockQuantity=0	Open its detail page	stockQuantity=0	'Out of stock' label shown; Add to cart disabled	—	Pass / Fail
TC-CAT-11	Unit	FR-005	Detail response contains all required fields	Book exists	Call CatalogService.getByld()	Seeded book id	Response has title, author, category, price, stockQuantity/stock status, description, coverImageUrl	—	Pass / Fail
TC-CAT-12	Integration	FR-005, SEC-003	Malformed book id handled safely	None	GET /books/abc123	id=abc123	422 or 404 with standard error envelope; no server crash or stack trace	—	Pass / Fail
TC-CAT-13	System	FR-005	Detail page on mobile viewport and missing cover image	Book without cover image	Open detail page at 375 px width	Book with empty coverImageUrl	Layout stays readable; placeholder image shown instead of a broken image	—	Pass / Fail
TC-CAT-14	Integration	FR-005, FR-013	Displayed price matches database after admin edit	Admin changed price to 449.00	GET /books/{id}	price=449.00	Detail shows 449.00 with two decimal places	—	Pass / Fail

Test Case ID	Level	Requirement	Description	Precondition	Steps	Test Data	Expected Result	Actual	Result
TC-CAT-15	System	FR-005, FR-006	Add to cart from detail page (logged in / logged out)	Book in stock	Click Add to cart once logged in; once logged out	In-stock book	Logged in: item added; logged out: redirected to login, then returned to the book	—	Pass / Fail
TC-CART-01	Unit	FR-006	Add an in-stock book to cart	Logged in; stock=5	CartService.addItem(bookId, 1)	qty=1	Cart has 1 line with quantity 1	—	Pass / Fail
TC-CART-02	Unit	FR-006	Reject out-of-stock book	Book stock=0	Add book to cart	qty=1	409 OUT_OF_STOCK; cart unchanged	—	Pass / Fail
TC-CART-03	Integration	FR-006, SEC-004	Cart requires authentication	Not logged in	POST /cart/items without token	No token	401 UNAUTHORIZED	—	Pass / Fail
TC-CART-04	Unit	FR-007	Update item quantity	Cart has 1 item	PATCH /cart/items/{bookId}	qty=3 (stock 5)	Line quantity becomes 3; total updated	—	Pass / Fail
TC-CART-05	Unit	FR-007	Quantity boundaries rejected	Cart has 1 item; stock=5	Set quantity to 0, -1, 6 and 5	qty=0, -1, 6, 5	0 and -1 give 422; 6 gives 409 (exceeds stock); 5 accepted	—	Pass / Fail
TC-CART-06	Unit	FR-007	Remove an item	Cart has 2 items	DELETE /cart/items/{bookId}	One bookId	Only the other item remains; total recalculated	—	Pass / Fail
TC-CART-07	Unit	FR-008	Cart total equals sum of price x quantity	Cart empty	Add 2 x 499.00 and 1 x 250.50; call calculateTotal()	499.00 x 2, 250.50 x 1	Total = 1248.50 (two decimal places)	—	Pass / Fail
TC-CART-08	Unit	FR-008	Empty cart total is zero	Cart empty	calculateTotal()	None	Total = 0.00	—	Pass / Fail
TC-ORD-01	Unit	FR-009	Reject checkout with empty cart	Logged in; cart empty	POST /orders	shippingAddress=Valid address	422 CART_EMPTY; no order, no payment call	—	Pass / Fail
TC-ORD-02	Unit	FR-009	Reject checkout with missing shipping address	Cart has 1 item	POST /orders without address	shippingAddress empty	422 VALIDATION_ERROR naming shippingAddress; no order	—	Pass / Fail
TC-ORD-03	Unit	FR-009, SEC-003	Shipping address length boundary	Cart has 1 item	POST /orders with 300 and 301 characters	300 chars / 301 chars	300 accepted; 301 rejected with 422	—	Pass / Fail
TC-ORD-04	Integration	FR-009, FR-010, FR-016	Successful checkout creates order	Cart total 1248.50; payment stub returns SUCCESS	POST /orders with Idempotency-Key	Key=K1; valid address	201; order PLACED, paymentStatus SUCCESS, orderNumber present; stock reduced; cart emptied	—	Pass / Fail

Test Case ID	Level	Requirement	Description	Precondition	Steps	Test Data	Expected Result	Actual	Result
TC-ORD-05	Integration	FR-010	Order numbers are unique	Two successful checkouts possible	Place two orders with different keys	Keys K1, K2	Two different orderNumber values	—	Pass / Fail
TC-ORD-06	Integration	FR-016	Declined payment creates no confirmed order	Payment stub returns DECLINED	POST /orders	Key=K3	402 PAYMENT_DECLINED; order CANCELLED with paymentStatus FAILED; stock restored; cart kept	—	Pass / Fail
TC-ORD-07	Integration	NFR-004, FR-016	Retry after payment timeout does not duplicate order	Payment stub times out on first call, succeeds on second	POST /orders (K4) then POST /orders again with K4	Key=K4	First call 504 PAYMENT_TIMEOUT with no order; retry produces exactly one order; a third call with K4 returns the same order (200)	—	Pass / Fail
TC-ORD-08	Integration	FR-009	Item sold out between cart and checkout	Cart holds last copy; another user buys it first	POST /orders	Key=K5	409 OUT_OF_STOCK; no payment attempted; no order	—	Pass / Fail
TC-ORD-09	Integration	FR-009, FR-010	Concurrent checkout of last copy	Stock=1; two customers have it in cart	Send both POST /orders simultaneously	Two users, different keys	Exactly one order succeeds; other gets 409; stock never negative	—	Pass / Fail
TC-ORD-10	System	FR-009, FR-010, FR-016	Place order end-to-end through UI	Logged in customer; seeded catalog	Search, add to cart, open cart, checkout, place order	Any in-stock book; valid address	Confirmation screen shows order number and status PLACED	—	Pass / Fail
TC-TRK-01	System	FR-012, FR-011	Order history lists own orders	Customer has 3 orders	Open Order History	None	Own 3 orders shown, newest first, each with status	—	Pass / Fail
TC-TRK-02	Integration	FR-012	View single order detail	Order exists	GET /orders/{orderId}	Own orderId	200; items, total, status and statusHistory shown	—	Pass / Fail
TC-TRK-03	Integration	FR-012, SEC-002	Customer cannot read another customer's order	Orders of customers A and B exist	As A, GET /orders/{B's orderId}	A's token; B's orderId	404 ORDER_NOT_FOUND; no order data leaked	—	Pass / Fail
TC-TRK-04	Unit	FR-011	Only allowed status values accepted	None	Create order with status 'SHIPPING'	Invalid enum	Model validation rejects; allowed values are PLACED, PROCESSING, SHIPPED, DELIVERED, CANCELLED	—	Pass / Fail
TC-TRK-05	Integration	Business rule 6.5	Customer cancels own PLACED order	Order PLACED	PATCH /orders/{id}/cancel	Own orderId	200; status CANCELLED; stock restored	—	Pass / Fail
TC-TRK-06	Integration	Business rule 6.5	Customer cannot cancel after PROCESSING	Order PROCESSING	PATCH /orders/{id}/cancel	Own orderId	409 CANCEL_NOT_ALLOWED; status unchanged	—	Pass / Fail

Test Case ID	Level	Requirement	Description	Precondition	Steps	Test Data	Expected Result	Actual	Result
TC-TRK-07	Integration	FR-012	Customer with no orders sees empty history	New customer	GET /orders	None	200 with empty list; UI shows 'No orders yet'	—	Pass / Fail
TC-TRK-08	Integration	FR-012, SEC-004	Order history requires authentication	Not logged in	GET /orders without token	No token	401 UNAUTHORIZED	—	Pass / Fail
TC-ADM-01	Integration	FR-013	Admin adds a book	Admin logged in	POST /admin/books	Valid title, author, category, price=399.00, stock=10	201; book appears in public catalog	—	Pass / Fail
TC-ADM-02	Integration	FR-013, SEC-003	Reject invalid book data	Admin logged in	POST /admin/books with missing title, negative price, title of 151 chars	title missing; price=-5; 151-char title	422 VALIDATION_ERROR for each; nothing saved	—	Pass / Fail
TC-ADM-03	Integration	FR-013	Admin edits a book	Book exists	PUT /admin/books/{id}	price=449.00	200; new price visible to customers	—	Pass / Fail
TC-ADM-04	Integration	FR-013	Admin disables a book	Book active	PUT /admin/books/{id} isActive=false	isActive=false	Book hidden from customer catalog and search; still listed in admin view	—	Pass / Fail
TC-ADM-05	Integration	FR-014	Admin updates stock	Book exists	PATCH /admin/books/{id}/stock with 25, then -1	25 / -1	25 accepted and shown on detail page; -1 gives 422	—	Pass / Fail
TC-ADM-10	System	FR-013	Admin catalog management end-to-end in UI	Admin logged in	Add a book, edit its title, disable it; check storefront each time	New book data	Storefront shows the book after add, shows the edited title, and hides it after disable	—	Pass / Fail
TC-ADM-11	Integration	FR-013, FR-009	Disabled book already in a cart is blocked at checkout	Book in a customer cart then disabled by admin	POST /orders	Key=K6	409 (book unavailable); no payment attempted; no order	—	Pass / Fail
TC-SEC-01	Security	SEC-002	Customer token blocked from admin catalog API	Customer logged in	POST /admin/books with customer token	Customer JWT	403 FORBIDDEN; unauthorised-access event logged; nothing created	—	Pass / Fail
TC-ADM-06	Integration	FR-015	Staff lists incoming orders	Orders in several statuses	GET /admin/orders? status=PLACED	status=PLACED	Only PLACED orders returned, with customer and items	—	Pass / Fail
TC-ADM-07	Integration	FR-015, FR-011	Valid status progression	Order PLACED	PATCH status PROCESSING, then SHIPPED, then DELIVERED	Three updates	Each returns 200; statusHistory records actor and time	—	Pass / Fail

Test Case ID	Level	Requirement	Description	Precondition	Steps	Test Data	Expected Result	Actual	Result
TC-ADM-08	Integration	FR-015, FR-011	Invalid transition rejected	Order DELIVERED	PATCH status PLACED; PATCH status PROCESSING on a CANCELLED order	PLACED / PROCESSING	409 INVALID_TRANSITION; status unchanged	—	Pass / Fail
TC-SEC-02	Security	SEC-002	Customer token blocked from status update	Customer logged in; order exists	PATCH /admin/orders/{id}/status with customer token	Customer JWT	403 FORBIDDEN; event logged; status unchanged	—	Pass / Fail
TC-ADM-09	System	FR-015, FR-011, FR-012	Customer sees status set by staff	Customer order PLACED; staff logged in on another browser	Staff sets SHIPPED; customer refreshes order history	Status=SHIPPED	Customer's order shows SHIPPED	—	Pass / Fail
TC-ADM-12	Integration	FR-015	Staff cancels a PROCESSING order	Order PROCESSING	PATCH /admin/orders/{id}/status CANCELLED	status=CANCELLED	200; status CANCELLED; stock restored; statusHistory updated	—	Pass / Fail
TC-ADM-13	Integration	FR-015	Status update on unknown order	Staff logged in	PATCH /admin/orders/{unknownId}/status	Random Objectid	404 ORDER_NOT_FOUND	—	Pass / Fail
TC-ADM-14	Integration	FR-015, FR-011	Invalid status value rejected	Order PLACED	PATCH status SHIPPING	status=SHIPPING	422 VALIDATION_ERROR; status unchanged	—	Pass / Fail
TC-SEC-03	Security	SEC-003	Script injection in text fields	Logged in customer	Submit script text as shipping address and search term	<script>alert(1)</script>	Stored/echoed as escaped text; no script runs; no server error	—	Pass / Fail
TC-SEC-04	Security	SEC-004	Tampered token rejected	Valid token obtained	Alter payload role to ADMIN_STAFF without re-signing; call GET /admin/orders	Modified JWT	401 INVALID_TOKEN; no data returned	—	Pass / Fail
TC-SEC-05	Review	SEC-005	No payment card data stored	Orders with payments exist	Inspect schemas, order documents and logs	Test orders	Only paymentStatus and paymentRef stored; no card number, CVV or expiry anywhere	—	Pass / Fail
TC-SEC-06	System	SEC-006	Security events are logged	Security log enabled	Perform 3 failed logins and 1 customer call to an admin route	Wrong password x3; customer token	4 entries with time, IP, route, user id if known; no passwords or tokens	—	Pass / Fail
TC-SEC-07	Security	SEC-006 (DoS control, SAD 3.9)	Login rate limit	Fresh IP	Send 11 login attempts within 15 minutes	11 requests	First 10 processed; 11th returns 429 TOO_MANY_REQUESTS	—	Pass / Fail
TC-PERF-01	Performance	NFR-001	Search response time under load	1,000 books seeded; JMeter plan ready	Run 50 concurrent users for 5 min on search requests	50 virtual users; mixed title/author/category queries	95th percentile response time <= 1.5 s; error rate < 1%	—	Pass / Fail

Test Case ID	Level	Requirement	Description	Precondition	Steps	Test Data	Expected Result	Actual	Result
TC-AVAIL-01	Operational review	NFR-002	Availability in operating window	Deployed test environment; monitor polling /health every minute	Review uptime log for the defined window	Operating window agreed with instructor	Availability >= 99% excluding planned maintenance	—	Pass / Fail
TC-USA-01	Usability	NFR-003	New customer places order in <= 5 screens	5 first-time users; no instructions	Ask each to find a book and buy it; count primary screens	Standard task script	All users finish within 5 primary screens (login, catalog, cart, checkout, confirmation)	—	Pass / Fail
TC-MNT-01	Review	NFR-005	Business logic separated into service modules	Source code available	Code review checklist: no business rules in routes/controllers; service unit tests exist and pass	Repository	Rules live in services; unit tests run in CI; target >= 70% line coverage on services	—	Pass / Fail
TC-TST-01	Unit	NFR-006	Order service testable without payment service	No network access	Run OrderService tests with PaymentAdapter stub returning SUCCESS, DECLINED, TIMEOUT	Stubbed adapter	All three outcomes testable offline; tests pass in CI	—	Pass / Fail

Appendix B: Test Data Matrix

Scenario	Data	Expected
Valid	Book in stock, qty=1, address of 40 characters	Order accepted (201)
Invalid	Unknown book id; empty cart; missing address	Rejected (404 / 422)
Boundary	Shipping address 300 characters; password 8 characters; qty equal to stock; limit=50	Accepted
Boundary	Shipping address 301 characters; password 7 characters; qty = stock + 1; qty 0; limit=51	Rejected (422 / 409)
Security	Customer token on /admin endpoints; expired or tampered JWT; script and operator payloads	403 / 401; no script runs; no injection
Failure	Payment stub returns DECLINED; TIMEOUT then SUCCESS on retry with same key	No confirmed order on decline; exactly one order after retry
Concurrency	Two users check out the last copy at the same time	One order, one 409; stock never negative

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Architecture and Design Specification

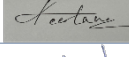
Project: Online Bookstore – Team 15

Project	Online Bookstore – Web-based Book Ordering & Catalog Management System
Project ID	BOOKSTORE-2026-T15
Version	1.0
Authors	Team 15: Anushka P (PES1202401127), B S Keertana (PES1202400463), Bhagath Naveen Mullakara (PES1202403272), Bhuvan Raj H (PES1202502988)
Date	01-10-2026
Status	Final
Course	UE24CS341A – Software Engineering
Technology Stack	MERN – MongoDB, Express.js, React, Node.js

Revision History

Version	Date	Author	Change Summary
1.0	01-10-2026	Team 15	Initial architecture and design specification, derived from SRS v1.0

Approvals

Role	Name	Signature/Date
Team Member (Auth)	Bhagath Naveen Mullakara	 2-10-2026
Team Member (Catalog & Search)	Anushka P	 02-10-2026
Team Member (Cart & Checkout)	B S Keertana	 02-10-2026
Team Member (Admin)	Bhuvan Raj H	 02-10-2026
Course Instructor / Evaluator		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Online Bookstore, a web application that lets a small independent or campus bookstore publish its catalog online and lets customers browse, search and purchase books. It describes how the requirements in SRS v1.0 are realised as a set of components, interfaces and interactions.

1.2 Scope

Covers the customer storefront (registration/login, catalog browsing and search, book details, cart, checkout with a mock payment, order tracking and history) and the Administrator / Staff dashboard

(catalog and stock management, order fulfilment). Out of scope: a real payment gateway, shipping-carrier integration, e-mail/SMS notifications, and recommendation features.

1.3 Audience

Developers (all four team members), QA/test engineers writing against the Software Test Plan, the course instructor and evaluators, security reviewers, and anyone maintaining the system later.

1.4 Definitions

Term	Meaning
SPA	Single-Page Application (the React frontend)
REST	Representational State Transfer; JSON-over-HTTPS API style
JWT	JSON Web Token used to authenticate API requests
RBAC	Role-Based Access Control (roles: CUSTOMER, ADMIN_STAFF)
ODM	Object-Document Mapper (Mongoose for MongoDB)
ADR	Architecture Decision Record
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
Idempotency key	Client-generated unique value that makes a repeated checkout request safe to retry
SRS / STP / RTM	Software Requirements Specification / Software Test Plan / Requirements Traceability Matrix

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture: goals, component view, chosen pattern, technology, risks, traceability to the SRS, and the security architecture. Section 4 describes the detailed design: module ownership and data model, UML sequence diagrams for three key flows, API design, error handling/logging, UX design and open issues. Identifiers used for traceability are: **C-nn** (components), **SD-n** (sequence diagrams), **API-nn** (endpoints), **ADR-nn** (decisions) and **SO-n** (security objectives).

2.2 Related Documents

Online Bookstore SRS v1.0 (FR-001 to FR-016, NFR-001 to NFR-006, SEC-001 to SEC-006); Online Bookstore Software Test Plan v1.0 (STP) and its requirements traceability matrix (RTM); Team 15 Synopsis v1.0; PES University Software Engineering Level 3 Submission Guide.

3. Architecture

3.1 Goals & Constraints

Goals (each is driven by an SRS requirement):

- **Security:** passwords hashed, JWT sessions, role separation, validated input, audit logging (SEC-001 to SEC-006).
- **Correctness of money and stock:** no duplicate order on payment retry and no confirmed order on payment failure (NFR-004, FR-016); stock never goes negative.
- **Performance:** 95% of catalog searches answered within 1.5 s at 50 concurrent users (NFR-001).
- **Availability:** at least 99% in the operating window (NFR-002).

- **Maintainability and testability:** business logic in service modules with automated unit tests, testable without the payment service (NFR-005, NFR-006).
- **Usability:** a new customer can place an order in no more than 5 primary screens (NFR-003).

Constraints: MERN stack is fixed; JWT-based sessions; payment is mock/simulated only; Agile with Git/GitHub, CI/CD and code reviews are mandatory; four-person team with a fixed course timeline, so a modular monolith is preferred over distributed services.

3.2 Stakeholders & Concerns

Stakeholder	Concerns	Addressed by
Customers	Easy ordering, accurate order status, safe credentials	SPA storefront, order state machine, SEC-001/004, ADR-03
Administrator / Staff (store)	Quick catalog/stock updates, clear order queue	Admin dashboard (C-02), Admin module (C-09)
Course instructor / evaluators	Traceability from requirement to test, documentation quality	Section 3.8 traceability, STP RTM
Developers	Clear module boundaries, parallel work, stable API contracts	Layered modules (Section 4.1), versioned API (Section 4.3)
Security reviewers	Threats identified and mitigated	Section 3.9 STRIDE and controls

3.3 Component (UML) Diagram

The system is a three-tier web application. The browser-hosted React SPA calls a Node.js/Express REST API over HTTPS; the API is organised into feature modules behind a common security middleware chain and reaches MongoDB only through a data access layer. The Payment Adapter is the only component that talks to the (simulated) external payment service. Dashed arrows are in-process dependencies between modules.

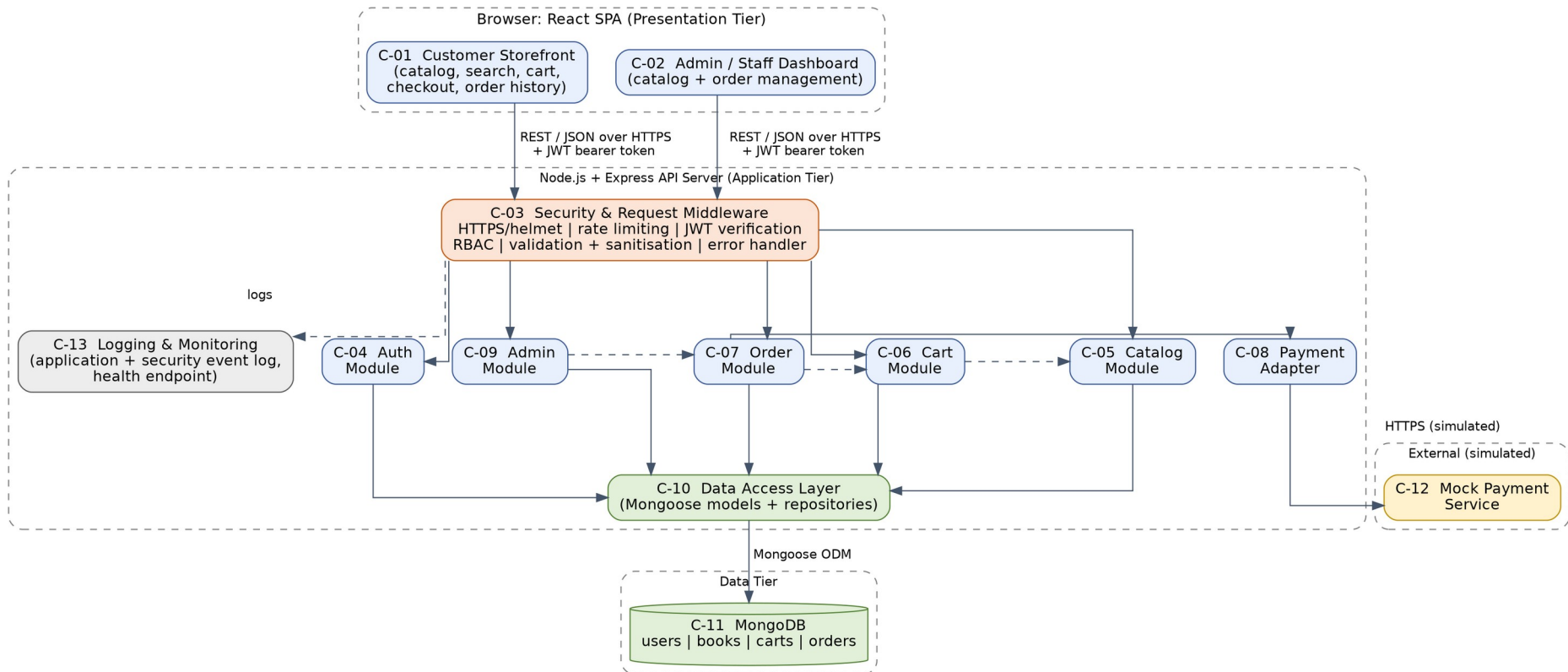


Figure 3.1 – Component diagram of the Online Bookstore (C-01 to C-13)

3.4 Component Descriptions

ID	Component	Responsibility	Interfaces / dependencies
C-01	Customer Storefront (React)	Catalog, search, book detail, cart, checkout and order-history screens; keeps JWT in browser session storage and attaches it to API calls.	Calls REST API (C-03) over HTTPS
C-02	Admin / Staff Dashboard (React)	Catalog management, stock update and order status screens; routes guarded by role.	Calls REST API (C-03) over HTTPS
C-03	Security & Request Middleware	helmet headers, CORS, rate limiting, JWT verification, RBAC check, request validation and sanitisation, central error handler, request logging.	Fronts every route; uses C-13 for logs
C-04	Auth Module	Registration, login, password hashing (bcrypt), JWT issue (role claim, 1 h expiry).	C-10 (users)
C-05	Catalog Module	Public browse, search by title/author/category, book detail.	C-10 (books)
C-06	Cart Module	Add, update, remove cart items; stock check; cart total calculation.	C-05 (price/stock), C-10 (carts)
C-07	Order Module	Checkout orchestration, unique order number, idempotency, stock reservation, order history, customer cancel, order state machine.	C-06, C-08, C-10 (orders, books)
C-08	Payment Adapter	Wraps the payment service behind a single interface (authorize, lookup); maps SUCCESS / DECLINED / TIMEOUT; stubbed in tests.	C-12 over HTTPS (simulated)
C-09	Admin Module	Admin catalog operations (add, edit, disable, stock) and fulfilment status updates.	C-10, C-07 (state machine)
C-10	Data Access Layer	Mongoose schemas, repositories, indexes (unique email, unique order number, unique customer+idempotency key).	C-11
C-11	MongoDB	Persistent storage for users, books, carts, orders.	Mongoose ODM
C-12	Mock Payment Service	Returns SUCCESS, DECLINED or delayed (timeout) results; idempotent per payment reference; never receives or stores card data.	Called by C-08
C-13	Logging & Monitoring	Structured application and security-event logs (no secrets), health endpoint, failure-rate metrics.	Used by C-03 and all modules

3.5 Chosen Architecture Pattern and Rationale

Pattern: layered (three-tier) architecture implemented as a modular monolith, with MVC-style separation inside the API (routes/controllers → services → repositories). The presentation tier is the React SPA; the application tier is the Express API; the data tier is MongoDB. Each layer talks only to the layer directly below it.

Why: it gives clear separation of concerns (NFR-005), lets four developers own separate modules with few merge conflicts, keeps business rules in services that can be unit-tested with mocked repositories and a stubbed payment adapter (NFR-006), and is simple to deploy and monitor for the expected load. Microservices were rejected as over-engineered for a small-bookstore workload and a four-person team.

ADR	Decision	Alternatives considered	Reason
ADR-01	Layered modular monolith	Microservices	Simplicity, team size, single deployable Docker image

ADR	Decision	Alternatives considered	Reason
ADR-02	Stateless JWT bearer tokens (1 h expiry, role claim)	Server-side sessions	Required by SRS 2.5; scales without session store; enforced by C-03
ADR-03	Idempotency-Key header on POST /orders, unique (customer, key) index	Client-side double-click guard only	Guarantees NFR-004 even after payment timeout and retry
ADR-04	Atomic conditional stock decrement (stock $\geq$ qty) before payment; release on failure	Check-then-write; decrement after payment	Prevents overselling under concurrent checkouts
ADR-05	Payment behind an adapter interface	Direct calls from Order module	Mock now, replaceable later; satisfies NFR-006 and SEC-005
ADR-06	Order items store a price/title snapshot; cart items are references	Reference-only order items	Order history stays correct if a price later changes

3.6 Technology Stack & Data Stores

Area	Choice
Frontend	React (Vite), React Router, fetch/axios, responsive CSS
Backend	Node.js, Express; jsonwebtoken, bcrypt, express-validator, express-mongo-sanitize, helmet, express-rate-limit, winston/morgan
Data store	MongoDB (local or Atlas) through Mongoose; collections: users, books, carts, orders
Transport	HTTPS; JSON; JWT in the Authorization: Bearer header
Testing	Jest + Supertest (backend), React Testing Library (frontend), Postman, JMeter
Delivery	Git/GitHub, GitHub Actions CI, Docker, ESLint, SonarQube or similar

3.7 Risks & Mitigations

Risk	Mitigation
Duplicate order after payment timeout and retry (NFR-004)	Idempotency key plus unique index; payment adapter is idempotent per reference (ADR-03)
Overselling the last copy under concurrent checkouts	Atomic conditional stock decrement; concurrency integration test (ADR-04)
Token theft or XSS exposing the JWT	Short expiry, helmet/CSP, output escaping in React, input sanitisation, no tokens in URLs or logs
API contract drift between four module owners	Versioned /api/v1 contract in Section 4.3; changes only via reviewed pull request; Postman collection run in CI
Database unavailable (single MongoDB instance)	Health endpoint and uptime monitoring; Atlas managed backups; graceful 503 responses
Schedule pressure across six sprints	Vertical-slice ownership, incremental delivery, CI gate on tests

3.8 Traceability to Requirements

Every functional, non-functional and security requirement maps to at least one component and one design element. The STP carries the same mapping through to test case IDs (STP Section 13).

Requirement	Component(s)	Design element
FR-001 Register	C-04, C-10	API-01; password hashing; unique email index
FR-002 Login	C-04, C-03	API-02; SD-3

Requirement	Component(s)	Design element
FR-003 Browse without login	C-05	API-03 (public route)
FR-004 Search	C-05, C-10	API-03 (q, category params); text/regex-safe query
FR-005 Book detail	C-05	API-04
FR-006 Add to cart (in-stock)	C-06, C-05	API-06
FR-007 Update / remove cart item	C-06	API-07, API-08
FR-008 Cart total	C-06	Cart.calculateTotal() in Cart service; API-05
FR-009 Valid checkout only	C-07, C-03	API-09; SD-1 steps 3 and 7
FR-010 Unique order number	C-07, C-10	orderNumber generator; unique index; SD-1 step 12
FR-011 Order status values	C-07, C-09	Order state machine (Figure 4.1); SD-2
FR-012 Order history and status	C-07	API-10, API-11
FR-013 Add / edit / disable books	C-09, C-05	API-13, API-14 (isActive flag)
FR-014 Update stock	C-09	API-15
FR-015 View orders, update fulfilment status	C-09, C-07	API-16, API-17; SD-2
FR-016 Payment result recorded, no confirm on failure	C-07, C-08, C-12	SD-1 (SUCCESS / DECLINED / TIMEOUT branches)
NFR-001 Search performance	C-05, C-10	Indexes on title, author, category; pagination (limit ≤ 50)
NFR-002 Availability	C-13, deployment	Health endpoint, uptime monitoring, Docker restart policy
NFR-003 ≤ 5 screens to order	C-01	Section 4.5 screen flow
NFR-004 No duplicate order on retry	C-07, C-08, C-10	Idempotency-Key (ADR-03); SD-1
NFR-005 Maintainability	All modules	Layering (Section 3.5); service-level unit tests
NFR-006 Testability	C-07, C-08	Payment adapter interface and stub (ADR-05)
SEC-001 Password hashing	C-04	bcrypt, 60-char hash field
SEC-002 RBAC	C-03, C-09	RBAC middleware on /admin routes; SD-2
SEC-003 Validate and sanitise input	C-03	express-validator, mongo-sanitize, length limits per Appendix B
SEC-004 Reject invalid/expired JWT	C-03	JWT verification middleware; 401 handling
SEC-005 No unnecessary payment data	C-07, C-08, C-12	Only paymentStatus and reference stored; no card fields in schema
SEC-006 Security event logging	C-13, C-03	Failed-login and unauthorised-access events (SD-2, SD-3)

3.9 Security Architecture

Security objectives

ID	Objective	Related requirements
SO-1	Confidentiality of credentials: user passwords can never be recovered from the database or logs.	SEC-001, SEC-005
SO-2	Authenticity of requests: only holders of a valid, unexpired token can use protected APIs.	SEC-004

ID	Objective	Related requirements
SO-3	Authorisation (least privilege): customers can reach only their own cart and orders; catalog and fulfilment functions are limited to Administrator / Staff.	SEC-002
SO-4	Integrity of data: untrusted input cannot alter queries, scripts or stored data; orders and payment results cannot be duplicated or forged.	SEC-003, NFR-004
SO-5	Accountability and detection: security-relevant events are recorded so misuse can be detected and investigated.	SEC-006
SO-6	Data minimisation: the application never receives or stores real payment card data.	SEC-005

Trust boundaries

(1) Browser to API (public internet, HTTPS only); (2) API to MongoDB (private network or Atlas with TLS and credentials); (3) API to payment service (simulated external system). All data crossing boundary 1 is untrusted until validated by C-03.

Threat model (STRIDE) and mitigations

Threat	Example in this system	Mitigation	Requirement
Spoofing	Attacker logs in as another user or forges a token	bcrypt-verified login, signed JWT with expiry, generic login error, login rate limit (10 attempts / 15 min / IP)	SEC-001, SEC-004
Tampering	Modified token payload (role = ADMIN_STAFF); NoSQL injection in search; changed price in checkout request	Signature verification; sanitise operators; server recomputes total from DB prices and ignores client totals	SEC-003, SEC-004
Repudiation	User denies placing or an admin denies cancelling an order	Order statusHistory with actor and time; security and audit logging	SEC-006
Information disclosure	Customer reads another customer's order; password hash or token appears in logs or errors	Ownership check on every order query (404 if not owner); no secrets in logs; generic error messages; TLS	SEC-002, SEC-005, SEC-006
Denial of service	Request flood on login or search	express-rate-limit, pagination limits, request body size limit	SEC-006, NFR-001
Elevation of privilege	Customer calls /admin endpoints	RBAC middleware on all /api/v1/admin routes; server-side checks (never UI-only)	SEC-002

Security controls summary

- **Transport:** HTTPS only; HSTS and security headers through helmet.
- **Credentials:** bcrypt with salt (cost factor 10 or higher); JWT secret kept in environment variables, never in the repository.
- **Session:** stateless JWT, 1 h expiry, role claim; client sends it in the Authorization header; invalid or expired tokens return 401.
- **Input handling:** schema validation with field lengths from SRS Appendix B (for example shipping address ≤ 300 characters); Mongo operator stripping; React escapes output by default.
- **Payment:** only paymentStatus and an opaque paymentRef are stored; no card number, CVV or expiry fields exist in any schema.
- **Logging:** failed logins and unauthorised access attempts are logged with time, IP, user id (if known) and route; passwords, tokens and full request bodies are never logged.

- **Pipeline:** dependency audit (npm audit), ESLint security rules and static analysis in CI; secrets scanning on the repository.

4. Design

4.1 Design Overview

The backend is a layered modular monolith: each module has routes/controllers (HTTP concerns), a service (business rules) and a repository (Mongoose). Controllers never contain business logic. The React SPA is split into a customer area and an Administrator / Staff area with role-guarded routes.

Module decomposition and ownership

Module	Owner	Requirements	Depends on
Catalog & search (C-05)	Anushka P	FR-003, FR-004, FR-005, NFR-001	Data access
Cart & checkout (C-06, C-07, C-08)	B S Keertana	FR-006 to FR-012 (customer side), FR-016, NFR-004, NFR-006	Catalog, Data access, Payment adapter
Authentication (C-04) and security middleware (C-03)	Bhagath Naveen Mullakara	FR-001, FR-002, SEC-001 to SEC-004	Data access
Admin catalog & order management (C-09, C-02)	Bhuvan Raj H	FR-013, FR-014, FR-015, SEC-002	Catalog, Order state machine

Data model (Mongoose collections)

Collection	Key fields	Constraints / indexes
users	_id, name, email (≤ 100), passwordHash (60), role (CUSTOMER ADMIN_STAFF), shippingAddress (≤ 300 , optional until first checkout), createdAt	unique(email)
books	_id, title (≤ 150), author (≤ 100), category (≤ 50), price (Decimal128), stockQuantity (int ≥ 0), description, coverImageUrl, isActive	text index (title, author); index(category); stockQuantity ≥ 0
carts	_id, customerId, items[{bookId, quantity ≥ 1 }], updatedAt	unique(customerId)
orders	_id, orderNumber, customerId, items[{bookId, title, quantity, unitPrice}], total, shippingAddress (≤ 300), orderStatus (PLACED PROCESSING SHIPPED DELIVERED CANCELLED), paymentStatus (SUCCESS FAILED), paymentRef, idempotencyKey, statusHistory[{status, actor, at}], createdAt	unique(orderNumber); unique(customerId, idempotencyKey); index(customerId, createdAt)

Order state machine

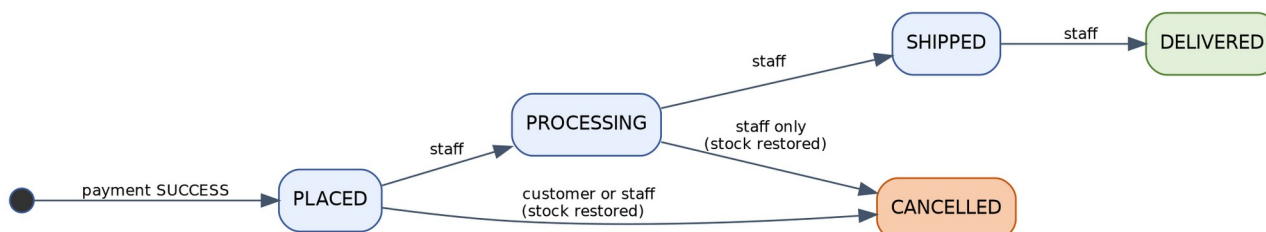


Figure 4.1 – Allowed order status transitions (any other transition returns 409 INVALID_TRANSITION)

A customer may cancel only while the order is PLACED (SRS business rule); staff may also cancel a PROCESSING order. Cancelling restores the stock reserved by the order.

4.2 UML Sequence Diagrams

Three flows specific to this project are specified: SD-1 Place Order (UC-05, the most critical flow), SD-2 Update Order Status with role enforcement (UC-08) and SD-3 Login (UC-01).

SD-1: Place Order / Checkout (UC-05; FR-009, FR-010, FR-016, NFR-004)

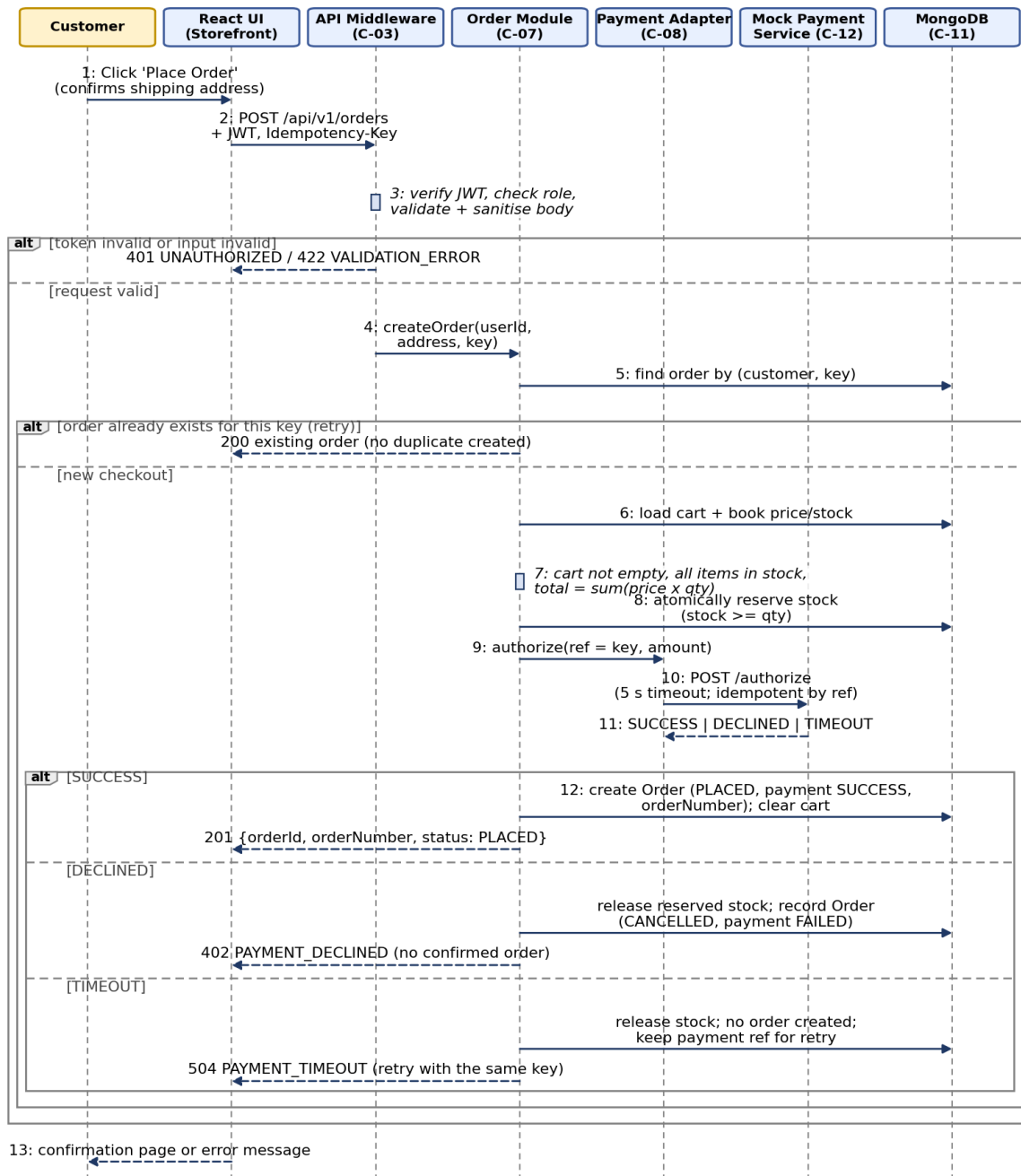


Figure 4.2 – Sequence diagram SD-1: Place Order with mock payment (success, declined, timeout, retry)

Notes. The server recomputes the total from database prices and ignores any client-supplied total. Stock is reserved atomically before payment and released on decline or timeout. The payment reference equals the idempotency key, so a retry after a timeout either finds the existing order (step 5) or re-asks the idempotent payment service, and can never create a second order.

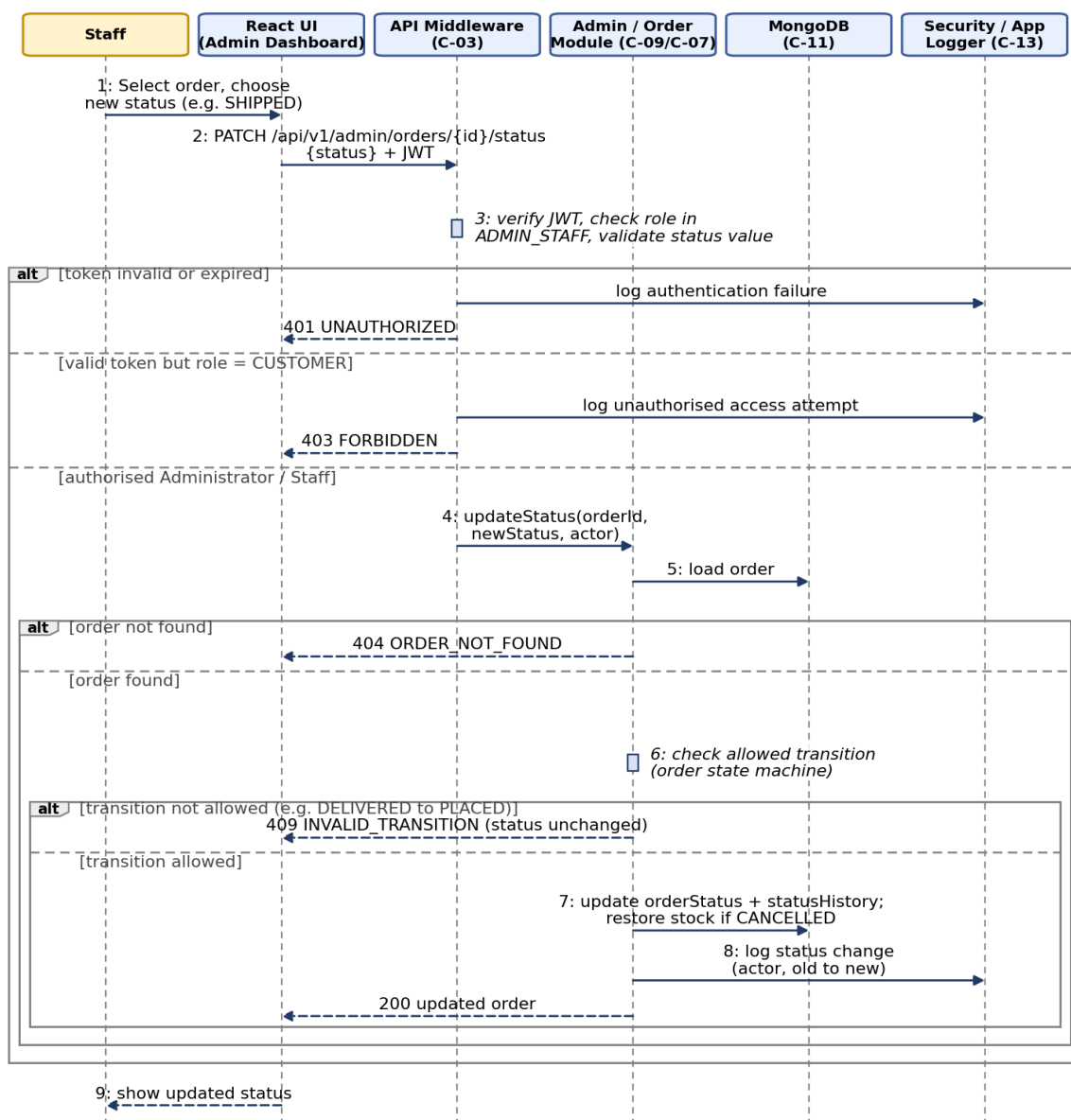
SD-2: Update Order Status by Administrator / Staff (UC-08; FR-011, FR-015, SEC-002, SEC-006)

Figure 4.3 – Sequence diagram SD-2: fulfilment status update with RBAC and state-machine enforcement

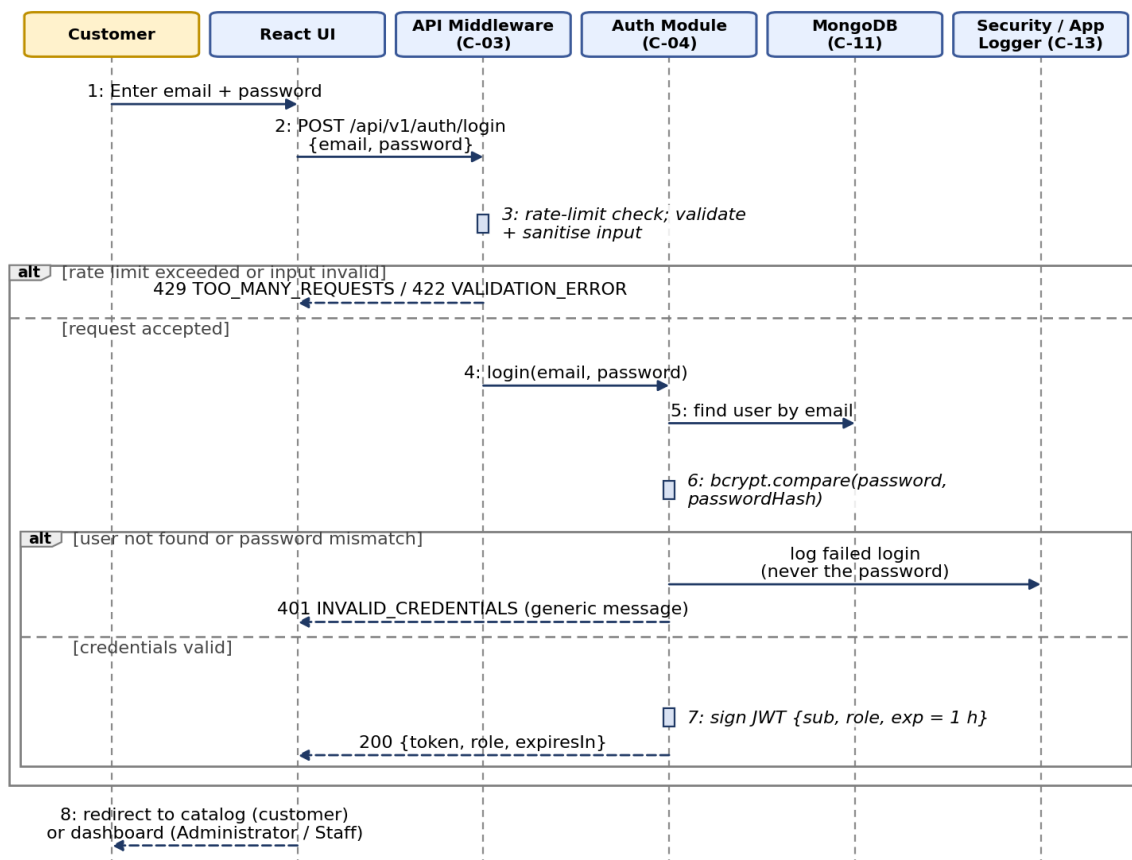
SD-3: Login (UC-01; FR-002, SEC-001, SEC-004, SEC-006)

Figure 4.4 – Sequence diagram SD-3: login with rate limiting, bcrypt verification and JWT issue

4.3 API Design

Conventions. Base path /api/v1 (version in the path; contracts are not changed silently once other modules depend on them). JSON request and response bodies. Protected endpoints require Authorization: Bearer <JWT>. Roles: **Public**, **CUSTOMER**, **ADMIN_STAFF**. All error responses use the standard envelope in Section 4.4.

Endpoint summary

ID	Method and path	Purpose	Auth	Success	Error codes
API-01	POST /auth/register	Create customer account	Public	201	409, 422
API-02	POST /auth/login	Authenticate, return JWT	Public	200	401, 422, 429
API-03	GET /books? q=&category=&page=&limit=	Browse and search active books	Public	200	422
API-04	GET /books/{bookId}	Book detail	Public	200	404
API-05	GET /cart	View cart with total	CUSTOMER	200	401
API-06	POST /cart/items	Add in-stock book	CUSTOMER	200	401, 404, 409, 422
API-07	PATCH /cart/items/{bookId}	Change quantity	CUSTOMER	200	401, 404, 409, 422
API-08	DELETE /cart/items/{bookId}	Remove item	CUSTOMER	200	401, 404
API-09	POST /orders	Checkout / place order	CUSTOMER	201 (200 replay)	401, 402, 409, 422, 504
API-10	GET /orders	Own order history	CUSTOMER	200	401
API-11	GET /orders/{orderId}	Own order detail and status	CUSTOMER	200	401, 404
API-12	PATCH /orders/{orderId}/cancel	Cancel own PLACED order	CUSTOMER	200	401, 404, 409
API-13	POST /admin/books	Add book	ADMIN_STAFF	201	401, 403, 422
API-14	PUT /admin/books/{bookId}	Edit or disable (isActive) book	ADMIN_STAFF	200	401, 403, 404, 422
API-15	PATCH /admin/books/{bookId}/stock	Set stock quantity	ADMIN_STAFF	200	401, 403, 404, 422
API-16	GET /admin/orders?status=	List incoming orders	ADMIN_STAFF	200	401, 403
API-17	PATCH /admin/orders/{orderId}/status	Update fulfilment status	ADMIN_STAFF	200	401, 403, 404, 409, 422
API-18	GET /health	Liveness check for monitoring	Public	200	503

Interface definition 1: Order component, POST /api/v1/orders (API-09)

Item	Definition
Auth	CUSTOMER JWT required
Headers	Authorization: Bearer <JWT>; Idempotency-Key: <UUID, required>; Content-Type: application/json
Request body	{ "shippingAddress": string, 1 to 300 characters, required } (cart contents are read server-side; any client total is ignored)

Item	Definition
Validation	Cart must not be empty; every item active and in stock; shippingAddress present and within length; Idempotency-Key present
201 Created	{ "orderId", "orderNumber", "orderStatus": "PLACED", "paymentStatus": "SUCCESS", "total", "items": [{bookId, title, quantity, unitPrice}], "createdAt" }
200 OK (replay)	Same body as 201 when the key already produced an order (no second order is created)
Errors	401 UNAUTHORIZED (missing/expired/invalid token); 422 VALIDATION_ERROR / CART_EMPTY; 409 OUT_OF_STOCK (details list the book ids); 402 PAYMENT_DECLINED (no confirmed order, cart kept); 504 PAYMENT_TIMEOUT (retry with the same key)

Interface definition 2: Auth component, POST /api/v1/auth/login (API-02)

Item	Definition
Auth	Public (rate limited: 10 attempts per 15 minutes per IP)
Request body	{ "email": string (valid format, ≤ 100), "password": string (8 to 72 characters) }
200 OK	{ "token": JWT, "role": "CUSTOMER" "ADMIN_STAFF", "expiresIn": 3600 }
Errors	401 INVALID_CREDENTIALS (same message for unknown email and wrong password); 422 VALIDATION_ERROR; 429 TOO_MANY_REQUESTS

Interface definition 3: Admin component, PATCH /api/v1/admin/orders/{orderId}/status (API-17)

Item	Definition
Auth	ADMIN_STAFF JWT required (customers receive 403 and the attempt is logged)
Request body	{ "status": "PROCESSING" "SHIPPED" "DELIVERED" "CANCELLED" }
200 OK	{ "orderId", "orderNumber", "orderStatus", "statusHistory": [{status, actor, at}] }
Errors	401 UNAUTHORIZED; 403 FORBIDDEN; 404 ORDER_NOT_FOUND; 409 INVALID_TRANSITION (see Figure 4.1); 422 VALIDATION_ERROR

Other contract rules

- Catalog search (API-03) returns { items: [...], page, limit, total } and caps limit at 50; query text is treated as data, never as a Mongo operator.
- Cart responses (API-05 to API-08) always return the full cart with per-line subtotal and the recomputed total.
- Book price fields are decimals with two places; stockQuantity is an integer ≥ 0 .

4.4 Error Handling, Logging & Monitoring

Standard error envelope

Every error uses one shape so the frontend can handle all failures uniformly:

```
{ "error": { "code": "OUT_OF_STOCK", "message": "Human-readable text", "details": [ ... ], "requestId": "uuid" } }
```

HTTP	Code(s)	When
400	BAD_REQUEST	Malformed JSON or missing required header
401	UNAUTHORIZED, TOKEN_EXPIRED, INVALID_CREDENTIALS	Missing/invalid/expired token or wrong login
402	PAYMENT_DECLINED	Mock payment service declined the payment

HTTP	Code(s)	When
403	FORBIDDEN	Valid token but role not permitted (RBAC)
404	BOOK_NOT_FOUND, ORDER_NOT_FOUND	Resource missing, disabled, or not owned by the caller
409	EMAIL_EXISTS, OUT_OF_STOCK, INVALID_TRANSITION, CANCEL_NOT_ALLOWED	State or uniqueness conflict
422	VALIDATION_ERROR, CART_EMPTY	Field fails validation (details name the field)
429	TOO_MANY_REQUESTS	Rate limit exceeded
500	INTERNAL_ERROR	Unexpected failure; generic message, full detail only in server log
503 / 504	SERVICE_UNAVAILABLE / PAYMENT_TIMEOUT	Database unavailable / payment service did not answer within 5 s

Handling rules

- A single central error-handling middleware converts thrown errors into the envelope; stack traces are never returned to clients.
- Payment failures never leave a confirmed order: decline records an order as CANCELLED with paymentStatus FAILED and releases stock; timeout creates no order and releases stock.
- Client side: the SPA shows field-level messages for 422, a retry prompt for 504 (reusing the same Idempotency-Key), and a re-login prompt for 401.

Logging and monitoring

- Structured JSON logs with requestId, route, status and duration; **no passwords, tokens, payment data or full request bodies** are logged.
- Security events (failed login, unauthorised access, invalid token) are written to a separate security log stream (SEC-006).
- Monitoring: /health endpoint polled for uptime (NFR-002), checkout failure rate, payment decline/timeout count, and p95 search latency (NFR-001).

4.5 UX Design

Responsive React UI for desktop and mobile browsers (Chrome, Firefox, Edge). To meet NFR-003, the basic purchase path uses at most five primary screens:

#	Primary screen	Purpose
1	Register / Login	Shown only when the customer is not authenticated (browsing needs no login)
2	Catalog and search results	Search box, category filter, book cards with Add to cart (book detail page is optional, reached from a card)
3	Cart	Edit quantities, remove items, see the total
4	Checkout	Shipping address and mock payment, Place Order button
5	Order confirmation	Order number and status; link to order history

- Administrator / Staff dashboard has two views: Catalog management (add, edit, disable, stock) and Order management (incoming orders, status update).
- Accessibility: readable font sizes, sufficient colour contrast, labelled form fields, keyboard-navigable controls, clear inline validation and error messages.
- Out-of-stock books show a clear label and a disabled Add to cart button; the checkout button is disabled while a request is in progress to avoid double submission.

4.6 Open Issues & Next Steps

- Confirm with the course faculty whether customer shipping address is captured at registration or at checkout (SRS Appendix B marks it mandatory on the customer record; this design collects it at checkout and saves it to the profile).
- Refresh tokens and logout-all are out of scope for v1.0; JWT lifetime is 1 hour.
- Final choices for the performance tool (JMeter) and static-analysis tool (SonarQube or similar) to be confirmed in Sprint 5.
- Possible future enhancements: real payment gateway, e-mail notifications, wishlists and reviews, multi-store support.

5. Appendices

5.1 Glossary

SPA, REST, JWT, RBAC, ODM, ADR, STRIDE, idempotency key, order state machine, mock payment (see Section 1.4 and SRS Appendix A).

5.2 References

- Online Bookstore SRS v1.0 (Team 15)
- IEEE 42010 (architecture description); IEEE 830 / ISO/IEC/IEEE 29148 (requirements)
- OWASP Top 10 and OWASP ASVS; OWASP REST Security Cheat Sheet
- NIST SP 800-160 (systems security engineering)

5.3 Tools

Graphviz and Python/matplotlib (diagrams in this document), draw.io / PlantUML (diagram editing), Postman and Swagger/OpenAPI (API contract), GitHub Projects/Jira (backlog).